

第3章 图形编程基础

内容提要：

- 3.1 GDI编程基础
- 3.2 OpenGL简介及工具包
- 3.3 OpenGL初步编程
- 3.4 OpenGL基本几何图形的绘制

3.1 GDI编程基础

- GDI是一种Windows提供的绘制图形和图像的函数库和类库
- 开发工具Visual C++的MFC类库中的部分类封装了大量的图形图像方法

3.1.1 GDI概述
3.1.2 MFC编程基础

版权所有人：聂烜

- 3.1.3 基本几何元素的绘制
- 3.1.4 图像的显示
- 3.1.5 二个综合应用实例

3.1.1 GDI概述

- ✓ GDI是Windows平台下生成、显示图形、图像的命令库，是应用程序开发的调用接口；
- ✓ GDI提供图像的读取、显示，及产生简单二维图形元素的命令；
- ✓ GDI是位于应用程序与不同硬件之间的中间层，这种结构让程序员从直接处理不同硬件的工作中解放出来，把硬件间的差异交给了GDI处理。通过GDI，应用程序可驱动不同输出设备特性，使Windows应用程序能够毫无障碍地在Windows支持的任何图形输出设备上运行；
- ✓ GDI是以文件的形式存储在系统中，系统需要输出图形时把它载入内存，如果转换成硬件命令时遇到非GDI命令，系统还可能载入硬件驱动程序，驱动程序辅助GDI把图形命令转换成硬件命令。
- ✓ Visual C++提供的类库MFC已经封装了主要的GDI函数，程序员可以在MFC程序中自由使用GDI提供的功能

3.1.1 GDI概述（续）

✓GDI的主要图形应用有：

✓二维游戏开发，如纸牌类、棋类；

✓影视特殊效果制作，如字幕制作；

✓开发可视化仿真类软件；

✓高级界面制作

版权所有人：聂烜



二维领域

3.1.2 MFC编程基础

可以利用程序向导自动生成界面（三种类型），
对于一个单文档程序，将生成的类有：

- 1) 表示程序的CWinApp类
- 2) 表示主窗体的CMainFrame类
- 3) 表示显示区的CView类
- 4) 和硬盘交互的CDocument类
- 5) 代表版本对话框的CAboutDlg类

版权所有人：聂烜

3.1.2 MFC编程基础（续）

在表示显示区的CView类中，有一个用于刷新显示区的方法OnDraw(CDC* pDC)

其输入参数pDC是一个CDC类的对象，专门用于画图，它提供了各种画笔、画刷、填充颜色等

版权所有人：聂烜

3.1.3 基本图形元素的绘制

1) 画直线:

MoveTo, LineTo

2) 画矩形

Rectangle

3) 画椭圆

Ellipse

4) 画圆弧

Arc

5) 画多边形

Polygon

版权所有人：聂烜

3.1.3 基本图形元素的绘制（续）

1) 画直线:

```
CPoint MoveTo( int x, int y );
```

```
CPoint MoveTo( POINT point );
```

```
BOOL LineTo( int x, int y ),
```

```
BOOL LineTo( POINT point );
```

版权所有人：聂烜

3.1.3 基本图形元素的绘制（续）

2) 画矩形

```
BOOL Rectangle( int x1, int y1, int x2, int y2 );
```

```
BOOL Rectangle( LPCRECT lpRect );
```

版权所有人：聂烜

3.1.3 基本图形元素的绘制（续）

3) 画椭圆

```
BOOL Ellipse( int x1, int y1, int x2, int y2 );
```

```
BOOL Ellipse( LPCRECT lpRect );
```

版权所有人：聂烜

3.1.3 基本图形元素的绘制（续）

4) 画圆弧

```
BOOL Arc( int x1, int y1, int x2, int y2, int x3, int y3,  
int x4, int y4 );
```

```
BOOL Arc( LPCRECT lpRect, POINT ptStart, POINT  
ptEnd );
```

版权所有人：聂烜

3.1.3 基本图形元素的绘制（续）

5) 画多边形

BOOL Polygon (LPPOINT *lpPoints*, int *nCount*);

版权所有人：聂烜

lpPoints : 顶点列表
nCount : 定点数目

3.1.3 图形元素的填充

- ✓ 刷子的定义与使用
- ✓ 使用CDC类的函数画填充图形

版权所有人：聂烜

例如,下面代码画一个填充矩形

3.1.3 图形元素的填充（续）

```
CBrush NewBrush, *pOldBrush;  
NewBrush.CreateSolidBrush(RGB(255, 0, 0));  
pOldBrush=pDC->SelectObject(&NewBrush);  
pDC->FillRect(&rect);  
pDC->SelectObject(pOldBrush);  
NewBrush.DeleteObject();
```

版权所有：聂烜

3.1.4 图像的显示

✓CBitmap 类

✓CDC类的BitBlt方法

版权所有人：聂烜

3.1.4 图像的显示 (续)

```
CDC memDC;  
CBitmap bmp;  
CBitmap* pOldBmp = NULL;  
BITMAP bm;  
bmp.LoadBitmap(IDB_TEST);  
bmp.GetBitmap(&bm);  
int Width=bm.bmWidth, Height=bm.bmHeight;  
memDC.CreateCompatibleDC(pDC);  
pOldBmp = memDC.SelectObject(&bmp);  
pDC->BitBlt(0,0,Width,Height,&memDC,0,0,SRCCOPY);  
memDC.SelectObject(pOldBmp);  
memDC.DeleteDC();  
bmp.DeleteObject();
```

版权所有人：聂烜

3.1.4 二个综合应用实例

在这个应用程序例子中，我们实现了一个画泡泡的效果，涉及到了画圆、画矩形、图形填充等操作。

例1

在这个应用程序例子中，我们实现了一个猫捉老鼠的游戏，涉及到了图像显示、画矩形、图形填充等操作。

例2

3.2 OpenGL简介及工具包GLUT

OpenGL所具有的功能基本上涵盖了计算机图形学所要包括的各个方面的内容，首先介绍关于OpenGL的一些基本概念、组成及其简单的功能。

版权所有人：聂烜

- 3.1.1 [OpenGL概述](#)
- 3.1.2 [OpenGL的功能](#)
- 3.1.3 [OpenGL的组成](#)
- 3.1.4 [OpenGL应用工具包GLUT](#)

3.2.1 OpenGL概述

- ✓ OpenGL是由SGI公司设计的一套底层三维图形API。它不是一种编程语言，而是提供了一些预封装的函数，c语言可以调用这些函数。
- ✓ OpenGL是一个开放图形库，目前在Windows、MacOS、OS/2、Unix/X-Windows等系统下均可使用，因此具有良好的可移植性，同时调用方法简洁明了，深受好评，应用广泛。
- ✓ OpenGL (Open Graphics Library)是独立于操作系统和硬件环境的三维图形软件库。由于其开放性和高度的可重用性，目前已成为业界标准。很多优秀的软件都是以它为基础开发出来的，象著名的产品有动画制作软件3DMAX，Soft Image，VR软件和GIS软件等等

3.2.2 OpenGL的功能

OpenGL的主要功能：

1) 几何建模

建立模型

2) 坐标变换

几何变换

3) 颜色模式设置

4) 光照和材质设置

5) 图像功能

6) 纹理映射

7) 实时动画

动画

真实感图
形生成

版权所有：聂烜

3.2.3 OpenGL的组成

在微机版本中，OpenGL主要包括三个函数库：

- ✓OpenGL核心库：其中包含了OpenGL最基本的命令函数。这些函数都以`gl`为前缀。（115个）
- ✓OpenGL实用库：它是比OpenGL核心库更高一层的函数库，实用程序库中的所有函数都是由OpenGL的核心库函数编写。函数都以`glu`为前缀。（43个）
- ✓OpenGL辅助库：提供了一些基本的窗口管理函数、事件处理函数和简单的事件函数。函数都以`aux`为前缀。（31个）

3.2.4 OpenGL应用工具包GLUT

目前AUX编程辅助库已经很大程度上被GLUT库所取代了。GLUT也许不能满足所有的OpenGL应用，但从学习OpenGL的角度，它将是一个良好的开端。按照使用功能，GLUT中的函数可以分为以下几类：

1. 初始化和创建窗口

为了初始化并打开一个窗口，需要调用五个函数完成必要的任务。

- 1) `void glutInit(int argc, char**argv);` 该函数用于初始化GLUT库，其参数应与主函数`main()`的参数相同。应该在调用其他GLUT函数之前调用`glutInit()`函数。
- 2) `void glutInitDisplayMode(unsigned int mode);` 该函数为即将创建的窗口指定一种显示模式。参数的默认值为`GLUT_RGBA | GLUT_SINGLE`，即指定一个RGBA颜色模式的单缓存窗口。

3.2.4 OpenGL应用工具包GLUT (续)

- 3) `void glutInitWindowPosition(int x, int y);`
指定窗口左上角应该放置在屏幕上的位置。
- 4) `void glutInitWindowSize(int width, int height);`
指定了窗口以像素为单位的尺寸。
- 5) `void glutCreateWindow(char* string);`
创建一个允许使用的OpenGL窗口，并将其视为当前窗口。

版权所有人：聂烜

2. 处理窗口和输入

- 1) `void glutDisplayFunc(void(*func)(void));`
该函数用于绘制当前窗口。参数`void(*func)`为绘制当前窗口时所调用的函数名。任何时候当窗口的内容需要被重新绘制，则调用该函数。

3.2.4 OpenGL应用工具包GLUT(续)

2. 处理窗口和输入

- 2) `void glutReshapeFunc(void(*func)(int width, int height));`
表示在窗口尺寸改变时, 指定了所调用的函数。`width`, `height`指定了窗口新的宽度和高度。
- 3) `void glutKeyboardFunc(void(*func)(unsigned int key, int x, int y));`
参数`(*func)(unsigned int key, int x, int y)`为按下一个生成ASCII字符的键时, GLUT调用的函数名称。
- 4) `void glutMouseFunc(void(*func)(int button, int state, int x, int y));`该函数指定了当按下或释放一个鼠标键时, 调用的函数。参数`button`有三个有效值:
`GLUT_LEFT_BUTTON`, `GLUT_MIDDLE_BUTTON`以及
`GLUT_RIGHT_BUTTON` 分别代表鼠标的左键、中键和右键。

3.2.4 OpenGL应用工具包GLUT(续)

2. 处理窗口和输入

- 5) `void glutKeyboardFunc(void(*func)(unsigned int key, int x, int y));`

参数(*func)(unsigned int key, int x, int y)为按下一个非ASCII字符的键(如shift键)时, GLUT调用的函数名称。

- 6) `void glutMotionFunc(void(*func)(int x, int y));`

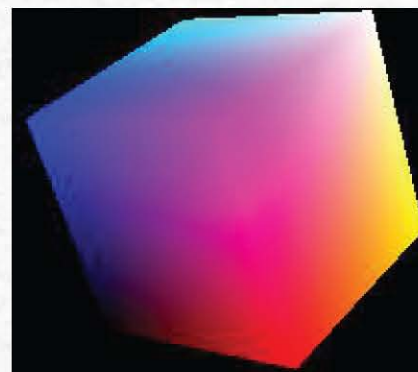
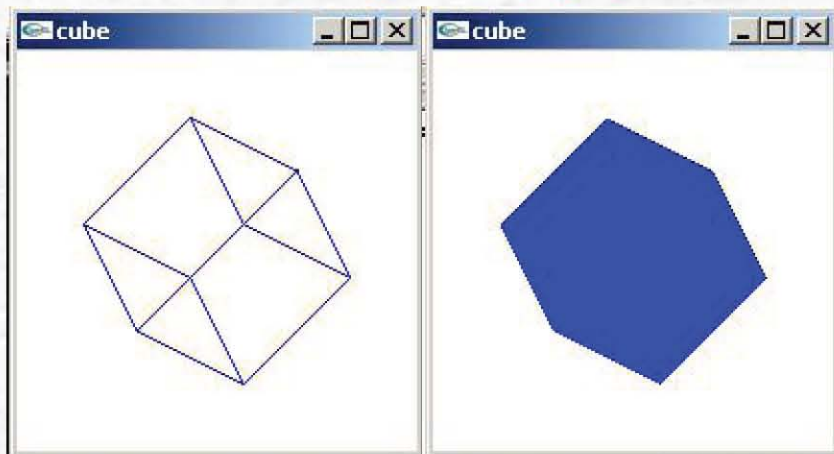
参数(*func)(int x, int y)为发生鼠标移动时, GLUT调用的函数名称。参数x和y返回鼠标当前的x和y位置。

3. 绘制三维物体

例如：绘制圆锥体，函数如下：

```
void glutWireCube(GLdouble size);  
void glutSolidCube(GLdouble size);  
可绘制九个三维物体
```

版权所有人：聂烜



4. 管理后台处理

当没有其他的待处理的事件时，用户可以用函数glutIdleFunc指定另一个函数。函数如下

```
Void glutIdleFunc*(void(*func)(void));
```

void(*func)指定了被调用的函数名。

版权所有人：聂烜

5. 运行程序

void glutMainLoop(void);这个函数开始启动主GLUT处理循环。事件循环包括所有的键盘、鼠标、绘制及窗口的事件等。

3.3 OpenGL编程初步

一个实用的OpenGL程序从规模上说比较庞大，但基本结构是非常简单的。一般包括以下几个部分：

- 1) 定义绘制有关的属性。
 - 定义窗口在屏幕上的位置及窗口的大小等属性。
 - 在窗口上建立坐标系，定义图形在窗口中的生成位置。
- 2) 初始化。
 - 初始化OpenGL中的状态变量，为图形显示作准备。
- 3) 渲染屏幕图像。
 - 按照显示的方位角度等要求绘制并显示图形
(将物体的数学描述及状态变量等变换为屏幕像素。)

主要内容

3.3.1 [OpenGL函数命名与数据类型](#)

3.3.2 [函数库和头文件](#)

3.3.3 [一个简单的OpenGL程序](#)

3.3.4 [窗口的坐标设置](#)

版权所有人：聂烜

3.3.1 OpenGL 函数命名与数据类型

OpenGL函数都遵循一个命名的约定：

例1 `glColor3f(1.0,1.0,1.0)` 命令，

- `gl`前缀代表该函数来自`gl`库（说明该函数来自哪个库）；
- 根命令`Color`代表对颜色的设定；
- 后缀`3f`表示该函数需要3个浮点型的参数；（后缀中的 数字表示该函数需要的参数个数，字母表示该函数的需要的参数类型）

版权所有人： 聂烜

例2 `GLfloat color[]={1.0,1.0,1.0}`
`glColor3fv(color);`

若函数的后缀中带有字母`v`，表示该命令带有一个指向矢量或数组值的指针作为参数。

3.3.2 库和头文件

- ✓编译OpenGL程序需要有头文件gl.h和glu.h、GLAUX.h, 库opengl32.lib, glu32.lib, GLAUX.lib,
- ✓若使用GLUT还需要glut.h 和glut32.lib。
- ✓运行OpenGL程序, 需要在windows\system目录下有动态连接库opengl32.dll和glu32.dll, GLAUX.dll,若使用GLUT还需要glut32.dll。

3.3.3 一个简单的OpenGL程序

一个简单OpenGL程序的清单如下：

```
#include <windows.h>
#include <gl/glut.h>
//绘图子程序
void display(void)
{ glClearColor(0.0f, 0.0f, 1.0f, 1.0f); // ...
  glClear(GL_COLOR_BUFFER_BIT); // ...
  glFlush(); // ...
}
//主程序
void main(int argc, char **argv)
{
  glutInit(&argc, argv);
  glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
  glutCreateWindow("hello");
  glutDisplayFunc(display);
  glutMainLoop();
}
```

版权所有人：聂烜

举例


```
void glClearColor(GLclampf red, GLclampf green,  
GLclampf blue GLclampf alpha);
```

这个函数设置清除窗口时所用的颜色。其前三个参数分别代表这种颜色中红、绿、蓝三种成分所占的比重。最后一个参数用于产生特殊效果，如半透明等。

```
void glClear(GL_COLOR_BUFFER_BIT);
```

用设定的清除颜色来清除窗口。

glFlush() 用于刷新OpenGL中的命令队列和缓冲区，使所有尚未被执行的OpenGL命令得到执行。即告诉OpenGL，应该处理到目前为止收到的命令。

绘制立方体的简单OpenGL程序的清单如下:

```
#include <windows.h>
#include <gl/glut.h>
void RenderScene(void)
{
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glColor4f(0.0, 0.0, 1.0, 1.0);
    glRotatef(60, 1.0, 1.0, 1.0);
    glutWireCube(0.8);
    glFlush();
}

//主程序
void main(int argc, char **argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(200, 200);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("cube");
    glutDisplayFunc(RenderScene);
    glutMainLoop();
}
```

设置作画颜色为蓝色

所绘图形绕一条从原点到(1.0,1.0,1.0)的线,沿逆时针方向旋转60度

版权所有人：聂烜

3.3.4 窗口坐标设置

本节介绍在窗口内设置作图区域及坐标系的有关内容。

✓在所有的窗口操作中，当执行初始化打开窗口、移动窗口或改变窗口的大小尺寸操作时，都会调用到GLUT中的一个函数：

```
void glutReshapeFunc(void(*func)(int width, int height));
```

它所包含的两个变量width和height，指定了窗口新的宽度和高度。我们就可以利用这些信息，定义OpenGL函数在窗口上坐标系及作图区域。

✓最常用的方法是调用函数glutReshapeFunc(myreshape)，其中myreshape()是用户自己定义的一个函数。

3.3.4 窗口坐标设置(续)

glutReshapeFunc (myseshape)

```
void myreshape(GLsizei w, GLsizei h)
{
    glViewport(0, 0, (GLsizei) w, (GLsizei) h); // ...
    glMatrixMode(GL_PROJECTION); // ...
    glLoadIdentity();
    gluOrtho2D(0.0, (GLdouble)w, 0.0, (GLdouble)h); // ...
}
```

版权所有人：聂烜

在窗口中
定义作图
区域

在世界坐
标系中定
义要显示
的区域

3.3.4 窗口坐标设置(续)

按下述方式定义的窗口坐标系统，物体的大小可以随着窗口的改变而进行缩放：

`glutReshapeFunc (myseshape)`

```
void myreshape(GLsizei w, GLsizei h)
{
    glViewport(0, 0, (GLsizei) w, (GLsizei) h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if (w <= h)
        gluOrtho2D(0.0, 200.0, 0.0, 200.0 * (GLfloat) h / (GLfloat) w);
    else
        gluOrtho2D(0.0, 200.0 * (GLfloat) h / (GLfloat) w, 0.0, 200.0);
}
```



Rect1.exe

✓函数glViewport()定义窗口内的作图区域（即视区）。函数如下：

```
void glViewport(GLint x, GLint y, GLsizei width, GLsizei height);
```

功能：设置窗口中的作图区域，用于OpenGL绘图。

参数说明：(x, y)为绘图区域左下角的坐标，参数width和height为绘图区宽度和高度的像素数，即视区的尺寸。

版权所有人：聂烜

✓例：若函数的参数取为：

```
glViewport(0, 0, (GLsizei) w, (GLsizei) h);
```

如果参数w和h是内部传给该函数以像素为单位的当前窗口的宽度和高度。则定义了整个窗口区域为OpenGL的作图区。

3.4 OpenGL中基本几何图形的绘制

OpenGL的基本几何元素有点、线、多边形。从根本上看，OpenGL绘制的所有复杂三维物体都是由一定数量的基本图形元素构成的，曲线、曲面分别是由一系列直线段，多边形近似得到的。

版权所有人：聂烜

3.4.1 点的绘制

3.4.2 线的限制

3.4.3 多边形的绘制

3.4.1 点的绘制

顶点的设置命令:

✓ OpenGL的点用一组称为顶点的浮点数定义。所有的内部运算都是按顶点是三维点进行的。即使用户设定的是二维的顶点，OpenGL也会通过自动增加一个值为0的z坐标。

✓ 在OpenGL中，顶点的设置命令为:

比如: `void glVertex3f(...)`

✓ `void glVertex{234}{sifd}{v}(TYPE cords)`

参数{234}:为输入的坐标值个数;

参数{sifd}:为输入坐标的数据类型;

参数(cords)是四维坐标(x,y,z,w)的缩写, 最少必须用二维坐标(x,y)。默认值z为0.0,w为1.0。

版权所有人: 聂烜

3.4.1 点的绘制(续)

所有的几何图最终都是通过一组有序顶点来描述的。OpenGL中有十种基本图元，从空间中绘制的简单的点到任意边数的封闭多边形。用`glBegin`命令可告诉OpenGL开始把一组顶点解释为特定图元。然后用`glEnd`命令结束该图元的顶点列表。

✓ `void glBegin(GLenum mode);`

此函数标志描述一个几何图元的顶点列表的开始。图元的类型由`mode`来决定。共有`GL_POINTS`,`GL_LINES`,`GL_LINE_STRIP`等十种图元

`void glEnd(void);`

此函数标志着顶点列表的结束。

版权所有人：聂烜

3.4.1 点的绘制(续)

点的绘制:

```
glBegin(GL_POINTS);  
    glVertex3f(0.0,0.0,0.0);  
    glVertex3f(5.0,5.0,5.0);  
glEnd();
```

在绘制一个点时，点的大小的默认值是一个像素。可以用函数 `glPointSize()` 来对点的大小进行修改。函数如下：

```
void glPointSize(GLfloat size);
```

该命令以像素为单位设置绘制点的大小。



Pointjx.exe

3.4.2 线的绘制

线属性

- ✓ `void glLineWidth(GLfloat width);`
以像素为单位设置线绘制的宽度。
- ✓ `void glLineStipple(GLint factor, GLushort pattern);`
指定点画模式（线型）。
 - `factor` 指定线型模式中每位的乘数。`factor`的值在 $[1, 255]$ 之间，缺省值为1。
 - `pattern` 用16位整数指定位模式。位为1时，指定要绘；位为0时，指定不绘。缺省时，全部为1。位模式从低位开始。

例如：模式0x3f07，二进制表示为：0011 1111 0000 0111，即是从低位起绘3个像素，不绘5个像素，绘6个像素和不绘2个像素来连成一条线。设factor为2，则绘或不绘的像素相应都乘上2。

利用如下命令定义上述线型：

```
glLineStipple(2, 0x3f07);
```

```
glEnable(GL_LINE_STIPPLE);
```

在定义线型后，必须用glEnable()命令激活线型。下图表示用不同的模式和重复因子绘线。

当不激活线型时：

pattern	factor
0xffff	1

去活线型时调用glDisable(GL_LINE_STIPPLE)。

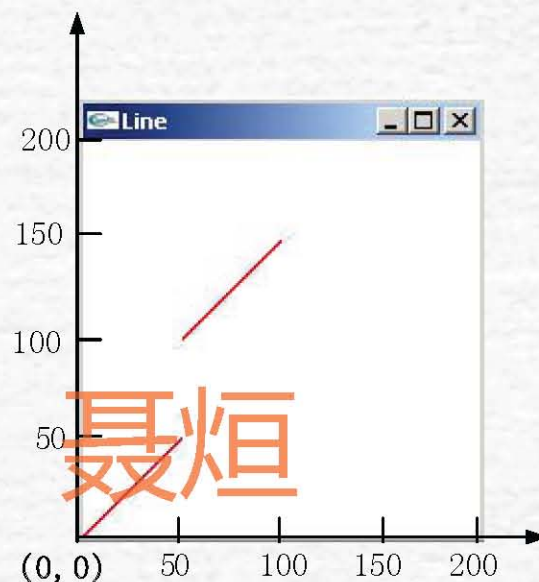
版权所有人：聂烜

PATTERN	FACTOR	
0x00FF	PATTERN	FACTOR
0x00FF	0x00FF	1
0x0C0F	0x00FF	2
0x0C0F	0x0C0F	1
0xAAAA	0x0C0F	3
0xAAAA	0xAAAA	1
0xAAAA	0xAAAA	2
0xAAAA	0xAAAA	3
	0xAAAA	4

版权所有：聂烜

独立线段的绘制:

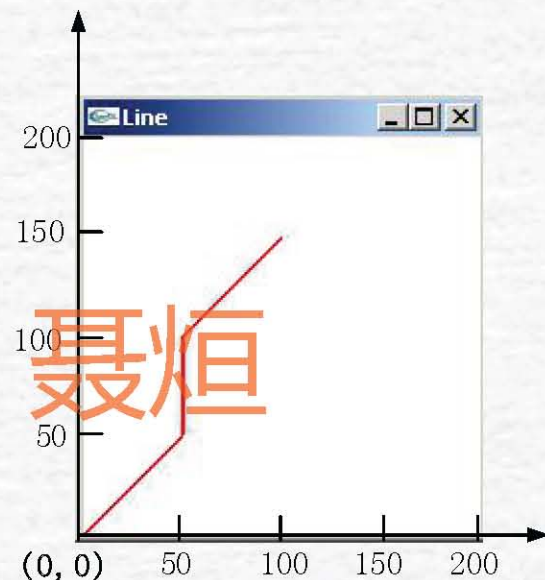
```
glColor3f(1.0,0.0,0.0);  
glLineWidth(5.0);  
glBegin(GL_LINES);  
    glVertex3f(0.0,0.0,0.0);  
    glVertex3f(50.0,50.0,0.0);  
    glVertex3f(50.0,100.0,0.0);  
    glVertex3f(100.0,150.0,0.0);  
glEnd();
```



3.4.2 线的绘制(续)

连接线段的绘制:

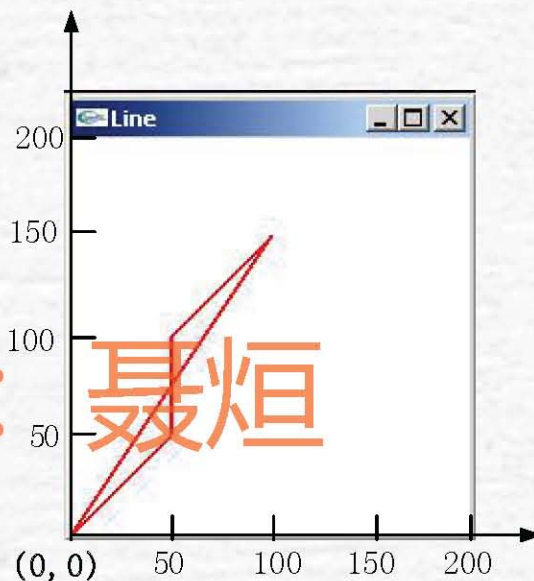
```
glColor3f(1.0,0.0,0.0);  
glLineWidth(5.0);  
glBegin(GL_LINE_STRIP);  
    glVertex3f(0.0,0.0,0.0);  
    glVertex3f(50.0,50.0,0.0);  
    glVertex3f(50.0,100.0,0.0);  
    glVertex3f(100.0,150.0,0.0);  
glEnd();
```



3.4.2 线的绘制(续)

闭合线段的绘制:

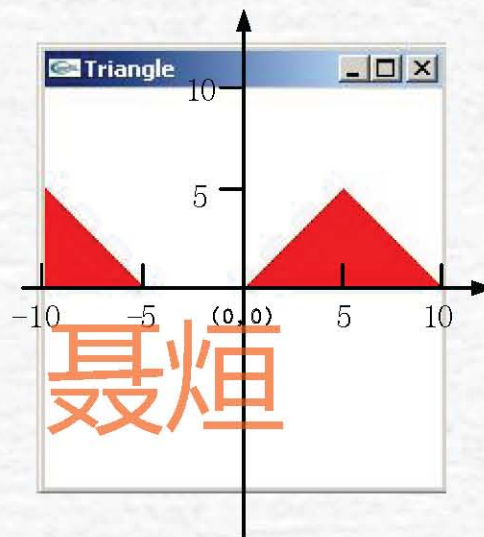
```
glColor3f(1.0,0.0,0.0);  
glLineWidth(5.0);  
glBegin(GL_LINE_LOOP);  
    glVertex3f(0.0,0.0,0.0);  
    glVertex3f(50.0,50.0,0.0);  
    glVertex3f(50.0,100.0,0.0);  
    glVertex3f(100.0,150.0,0.0);  
glEnd();
```



3.4.3 多边形的绘制

三角形的绘制:

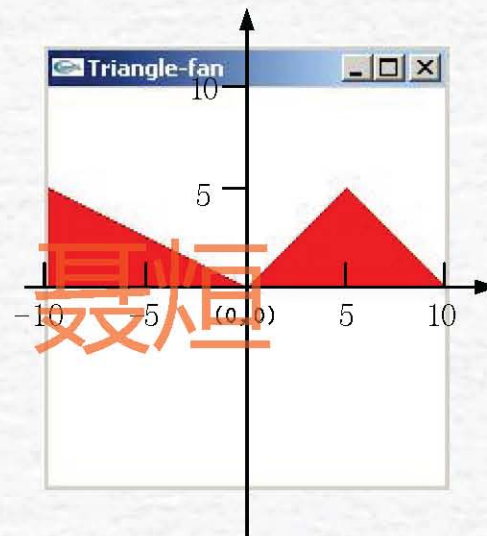
```
glColor3f(1.0,0.0,0.0);  
glBegin(GL_TRIANGLES);  
    glVertex3f(0.0,0.0,0.0);  
    glVertex3f(5.0,5.0,0.0);  
    glVertex3f(10.0, 0.0,0.0);  
    glVertex3f(-5.0, 0.0,0.0);  
    glVertex3f(-10.0,5.0,0.0);  
    glVertex3f(-10.0, 0.0,0.0);  
glEnd();
```



3.4.3 多边形的绘制(续)

相连三角形的绘制:

```
glColor3f(1.0,0.0,0.0);  
glBegin(GL_TRIANGLE_FAN);  
glVertex3f(0.0,0.0,0.0);  
glVertex3f(5.0,5.0,0.0);  
glVertex3f(10.0, 0.0,0.0);  
glVertex3f(-5.0, 0.0,0.0);  
glVertex3f(-10.0,5.0,0.0);  
glVertex3f(-10.0, 0.0,0.0);  
glEnd();
```



3.4.3 多边形的绘制(续)

四边形的绘制:

- ✓ OpenGL中的GL_QUADS图元用来绘制一个四边形。与三角形一样，GL_QUADS_STRIP图元指定一条相互连接的四边形。
- ✓ 由于矩形在图形应用中非常普遍，它不必通过glBegin和glEnd之间的顶点来进行绘制，而是通过使用如下的函数：
void glRect[sfd](TYPE x1, TYPE y1, TYPE x2, TYPE y2);

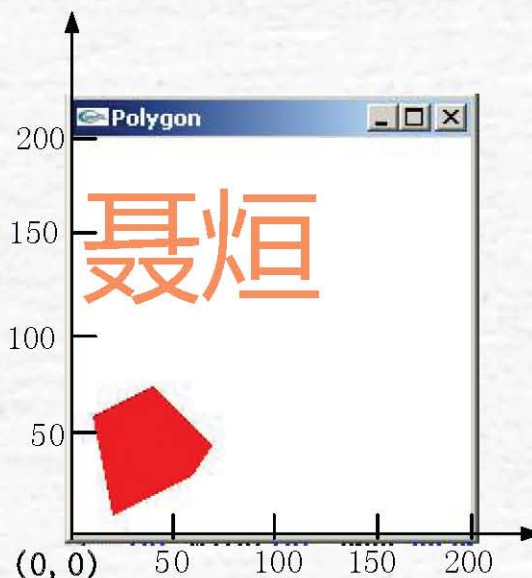
版权所有：聂烜

3.4.3 多边形的绘制(续)

多边形的绘制:

多边形的OpenGL图元是GL_POLYGON,可以用于绘制一个任意边数的多边形。要求多边形满足所有顶点都位于一个平面内,而多边形的各边不能相交。

```
glColor3f(1.0,0.0,0.0);  
glBegin(GL_POLYGON);  
    glVertex3f(20.0,10.0,0.0);  
    glVertex3f(60.0,30.0,0.0);  
    glVertex3f(70.0,45.0,0.0);  
    glVertex3f(40.0,75.0,0.0);  
    glVertex3f(10.0,60.0,0.0);  
glEnd();
```



3.4.3 多边形的绘制(续)

控制多边形的绘制:

在OpenGL中, 每个多边形被认为是由两个面组成的: 正面和反面。缺省时, 在屏幕上以逆时针方向出现顶点的多边形称为正面, 反之为背面。用户也可以利用函数`glFrontFace()`自行设置多边形的正面方向。

```
void glFrontFace(GLenum mode);
```

该函数定义多边形的正面方向。

mode: `GL_CW` (顺时针方向为正面);
`GL_CCW` (逆时针方向为正面, 缺省值)。

3.4.3 多边形的绘制(续)

选择绘制多边形的方式:

利用下面函数可以选择绘多边形的方式。

```
void glPolygonMode(GLenum face, GLenum mode);
```

face 控制多边形的正面和背面的绘图方式。

GL_FRONT_AND_BACK 正面和反面都画

GL_FRONT 只画正面

GL_BACK 只画反面

版权所有人：聂烜

3.4.3 多边形的绘制(续)

mode 控制绘点、线框或填充多边形。

GL_POINT 用有一定间隔的点填充

GL_LINE 只画多边形的边框

GL_FILL 填充多边形

缺省值: `glPolygonMode(GL_FRONT_AND_BACK, GL_FILL)`
即多边形的正、背面都用填充绘制。

例如, 下面两个函数的调用, 使得多边形正面填充, 背面是线框。

`glPolygonMode(GL_FRONT, GL_FILL);`

`glPolygonMode(GL_BACK, GL_LINE)。`

版权所有人：聂烜